



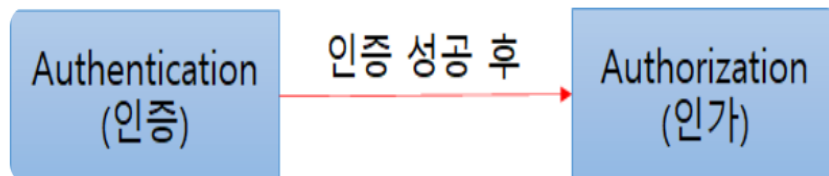
Spring Security

스프링FW 기반의 어플리케이션 보안(인증과 인가, 보안조치)을 담당하는 프레임워크

회원가입부터 로그인, 로그아웃, 세션 관리, 권한 관리까지 온라인 플랫폼에 맞춰 다양하게 작업되는 기능을 효과적으로 지원하기 위해 개발된 것이 Spring Security로 개발자들이 보안 관련 기능을 효율적이고 신속하게 구현할 수 있도록 도와준다.

사이트에 대해서 유효한 사용자인지 확인하는 것이 인증(로그인)이고, 인증된 사용자가 사용할 수 있는 기능인지 확인하고 권한이 부여된 리소스에만 접근을 허용하는 과정을 인가라고 한다.

Spring Security는 기본적으로 인증 절차를 거친 후에 인가 절차를 진행하게 되며, 인가 과정에서 해당 리소스에 대한 접근 권한이 있는지 확인하고 접근을 허용하게 된다.



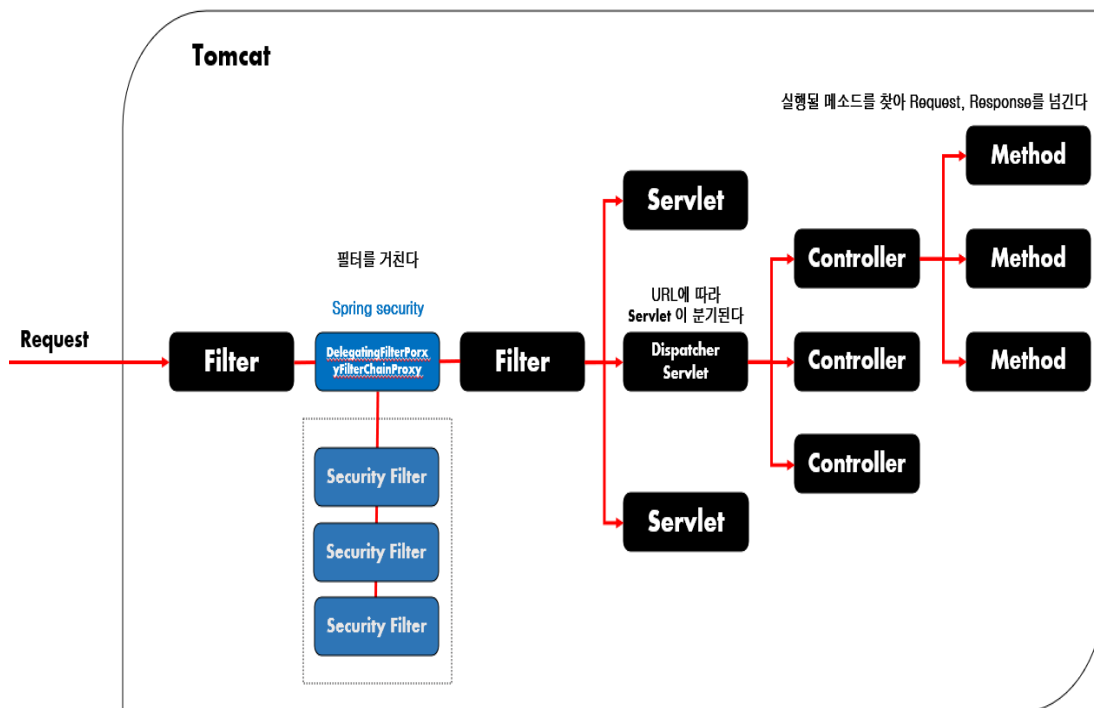
인증, 인가를 위해 Principal을 아이디로 Credential을 비밀번호로 사용하는 Credential 기반의 인증 방식을 사용한다.

Principal(접근 주체) : 보호 받는 리소스에 접근하는 대상

Credential(비밀번호) : 리소스에 접근하는 대상의 비밀번호

[Spring Security 특징]

스프링 시큐리티는 필터(Filter)를 기반으로 동작하기 때문에 스프링 MVC와 분리되어 관리 및 동작한다.



필터(Filter)는 서블릿(DispatcherServlet)으로 가기 전에 적용됨으로 가장 먼저 URL요청을 받는다.

세션&쿠키 방식으로 인증을 처리한다.

스프링의 DispatcherServlet앞단에 다양한 역할의 Filter로 구성되어 있다.

DispatcherServlet으로 넘어가기 전에 이 Filter가 요청을 가로채서 클라이언트의 리소스 접근 권한을 확인하고 없는 경우에는 인증 요청 화면으로 자동으로 리다이렉트 한다.

내장된 인증 과정은 폼로그인(/login)이다.

/logout 으로 요청하고 내장된 로그아웃도 제공한다.

용어

- 접근 주체(Principal) : 보호된 대상에 접근하는 사용자 객체
- 인증(Authentication) : 증명, 유저가 누구인지 확인하는 것, 회원가입하고 로그인하는 것
- 인가(Authorization) : 허락, 유저에 대한 권한을 확인, 허락하는 것

인증(Authentication)	인가(Authorization)
	
○ 시스템 접근 시, 등록된 사용자인지 여부를 확인하는 것 ○ 로그인	○ 인증 후 사용자에게 권한을 부여하는 것과 제한하는 것 ○ 사용자 등급(일반/VIP/관리자)

- 인증(Authentication)

: 현재 사용자가 누구인지 확인하는 과정으로 일반적인 웹 어플리케이션

아이디/암호를 이용해서 인증을 처리

- 1) 크리덴셜(Credential-자격) 기반 인증
- 2) 이중 인증(Two-factor authentication)
- 3) 물리적 인증

- 인가(Authorization)

: 현재 사용자가 접근하려는 자원에 대한 권한이 있는지 확인 절차가 필요한데, 특정 대상(URL, 기능)을 사용(접근)할 권한이 있는지 검사하는 절차.(항상 인증 선행 프로세스가 필요하다)

- 1) 부여된 권한(Granted Authority)
- 2) 리소스 권한(Intercept)

- 인증(Authentication)

종류	의미
크리덴셜(Credential :자격) 기반 인증	웹의 대부분의 인증방식. 권한을 부여 받기 위해 1차례의 인증 과정이 필요하며 ID와 비밀번호를 입력 받아 일치여부를 확인하는 것 스프링시큐리티에서는 아이디를 프린시플(principle) , 비밀번호를 크리덴셜(credential)이라고 부름
이중인증 (Two-factor authentication)	한번에 2가지 방식으로 인증 하는 것 ex) 로그인 + 보안증서 Authentication하나 추가됨으로서 프로그래밍 적으로 변화해야 할 부분이 상당히 광범위해짐.
물리적 인증	웹의 영역을 벗어난 것이지만 가장 효과적인 보안 수단 중에 하나로 컴퓨터를 켤 때 지문인식을 받거나 키를 삽입해야 하는 것.

- 인가(Authorization)

종류	의미
부여된 권한 (Granted Authority)	적절한 절차로 사용자가 인증되었다면 권한을 부여(Granted Authority) 해야 함 회원이입등을 통해 반 영구적인 권한이 부여되었다면 이 회원에게 부여된 권한을 어딘가에 저장 해야함.
리소스 권한 (Intercept)	보안이란 원래 권한이 없는 사용자들이 원천적으로 리소스에 접근할 수 없도록 하는 것이므로 적절한 권한을 가진 사용자만 자원을 접근할 수 있도록 자원의 외부 요청을 가로채는 것 (Intercept)이 웹 보안

보안을 위한 개발자들의 고민

- 개발자가 보안을 위해 구현 해야 할 일???

- ✓ 어떤 방식으로 권한을 부여할까?
- ✓ 해당 리소스에 어떻게 권한 수준을 부여할까?
- ✓ 인증 받은 사람은 어떻게 인증 받았다는 정보를 지속적으로 유지할 수 있을까?



Spring Security는 이러한 개발자의 고민을 한번에 해결해 줄 수 있는 프레임워크로 쉽고 편리할 뿐만 아니라 이런 보안 개념이 없는 개발자가 보안이 필요한 쇼핑몰이나 주요 웹사이트를 설계 했을 때 발생할 막대한 피해들을 방어 할 수 있는 최선의 선택임

인증 방식에는 어떤?

- 세션,쿠키 방식

: 사용자의 정보를 세션에 저장하여 서버에 관리

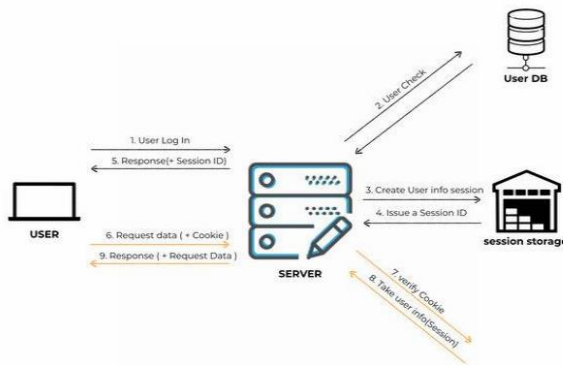
- 토큰(jwt - JSON WEB TOKEN) 방식

: 사용자의 정보를 JSON객체로 안전하게 전송하기 위한 방법

- 다른 채널을 통해 인증(Oauth)

: 로그인 open API

* 우리가 사용한 세션 - 쿠키 방식의 인증 흐름



1. 클라이언트가 서버로 로그인 요청을 보낸다.
2. 서버는 클라이언트가 보낸 (id, pw)를 확인한다.
3. (4 포함) 요청 정보가 유효하면 세션 ID를 생성한다.
4. 클라이언트는 응답으로 받은 세션을 쿠키에 저장한다.
5. 클라이언트가 인증이 필요한 요청을 할 때마다 헤더에 쿠키 실어서 보낸다.
6. 서버는 쿠키를 확인하여 사용자를 식별합니다.
7. 사용자에게 맞는 데이터를 넘겨줍니다.

* 그렇다면,

Spring security은?

: 세션-쿠키 방식을 사용한다.

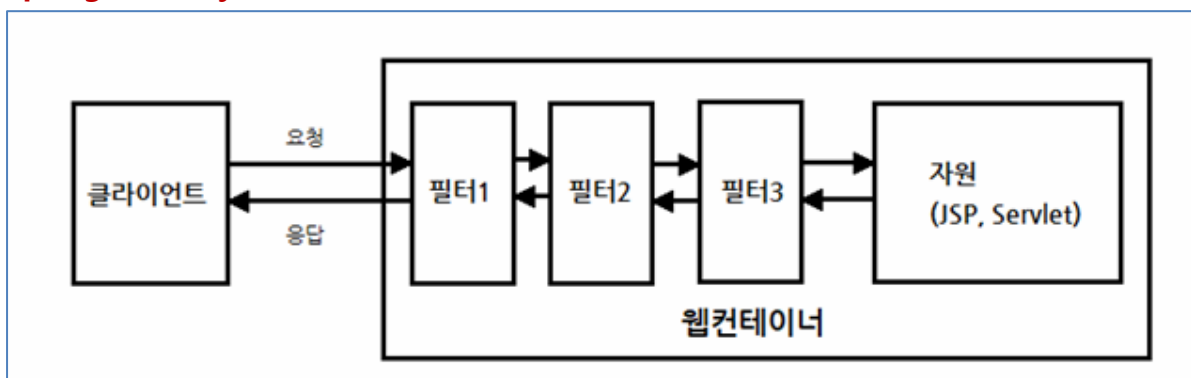
Spring Security 특징

: 오픈소스 프레임워크로 보편적인 인증, 인가, UI처리에 대한 기본 구현을 제공하고 있으며 일부 변경할 수 있게 확장이 가능하여 프로그래머는 처음부터 인증과 권한을 위한 코드를 만들기 보다는 스프링시큐리티가 제공하는 틀을 재사용하고 필요한 부분만 커스텀마이징 함으로써 보다 빠르게 인증과 권한 부분을 구현 할수 있음

: 암호화 기능도 제공 하여 암호화 기능에 대한 특별한 제약이 없다면 스프링 시큐리티가 제공하는 암호화 기능을 사용해서 비밀번호, 결제 정보등을 암호화할 수 있음

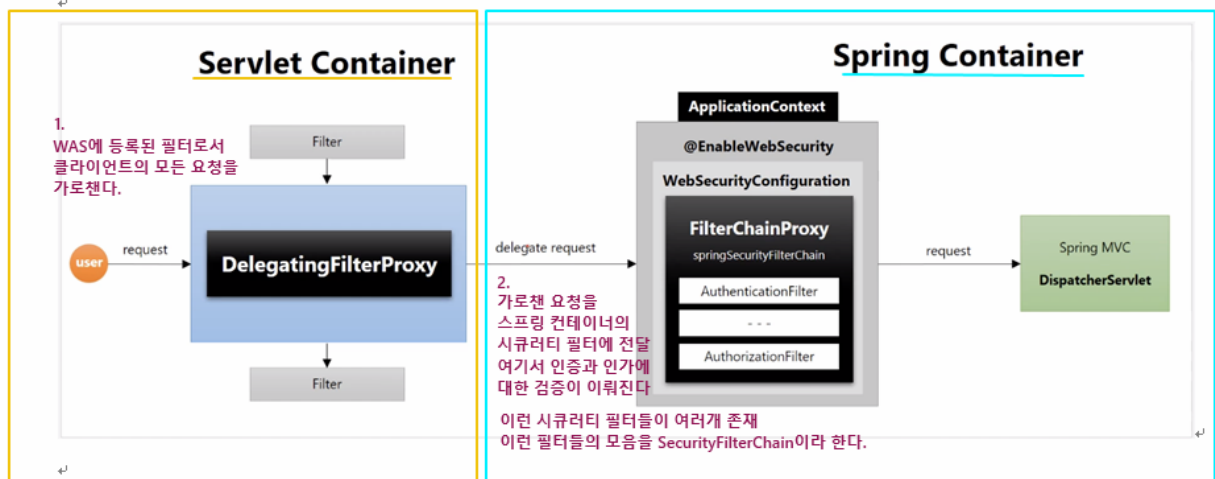
: in-Memory 기능 제공

Spring Security Filter



스프링 MVC에서는 요청을 가장 먼저 받는 것이 DispatcherServlet 이다.
이 DispatcherServlet이 요청을 받기 전에 다양한 필터들이 존재한다.

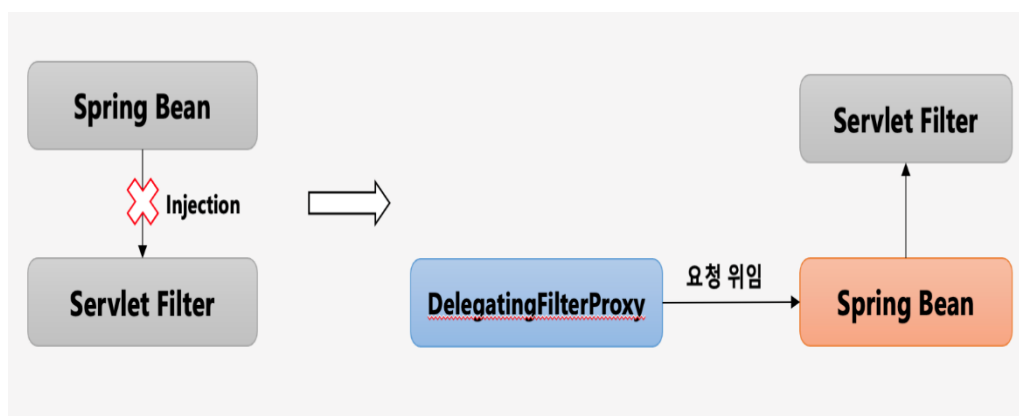
- ➔ 클라이언트의 요청은 여러 개의 필터를 거쳐서 서블릿(DispatcherServlet)으로 들어온다
- ➔ Tomcat WAS 위에서 가동되는 스프링부트 기반 컨테이너인 경우는 클라이언트의 요청이 WAS의 필터를 거쳐서 스프링 컨테이너 시큐리티필터를 다 거친 후 서블릿에 들어온다.

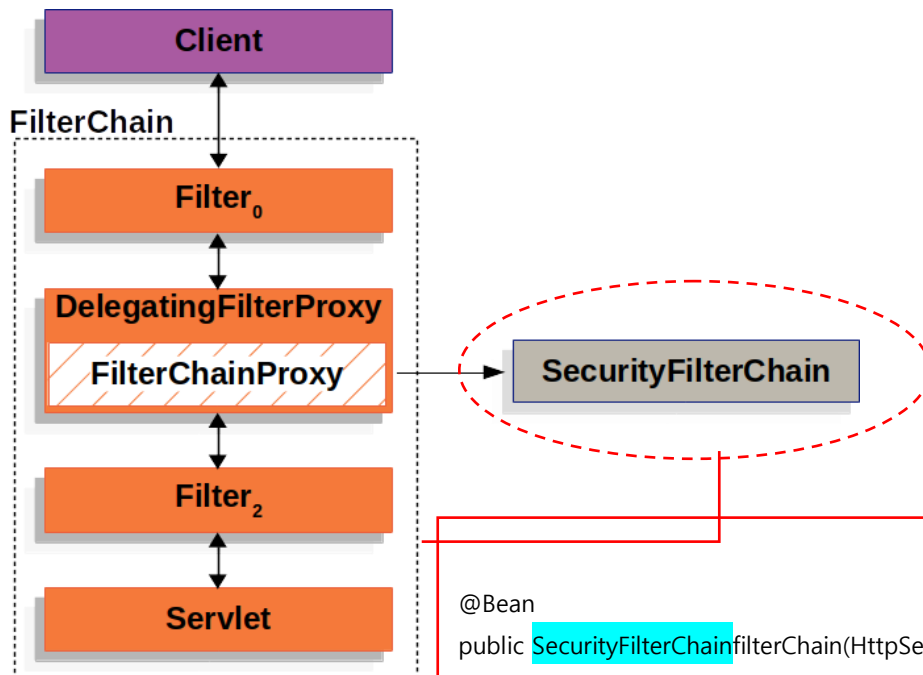


DelegatingFilterProxy (서블릿 필터와 스프링 빈의 연결 매개체)

Spring Security는 모든 요청에 대해서 필터를 기반으로 인증/인가처리를 한다.

@EnableWebSecurity 가 선언된 시큐리티관련 configuration 클래스에는 이런 저런 시큐리티관련 빈 객체들이 생성되는데 이 빈 객체들이 서블릿 필터에 영향을 주도록 역할을 하는 것이 바로 **DelegatingFilterProxy** 객체이다.





```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
    http
        .authorizeHttpRequests(requests -> requests
            .requestMatchers("/login", "/h2-console/**").permitAll()
            .requestMatchers("/todos/**").hasAuthority("ADMIN")
            .anyRequest().authenticated())
        :
  
```

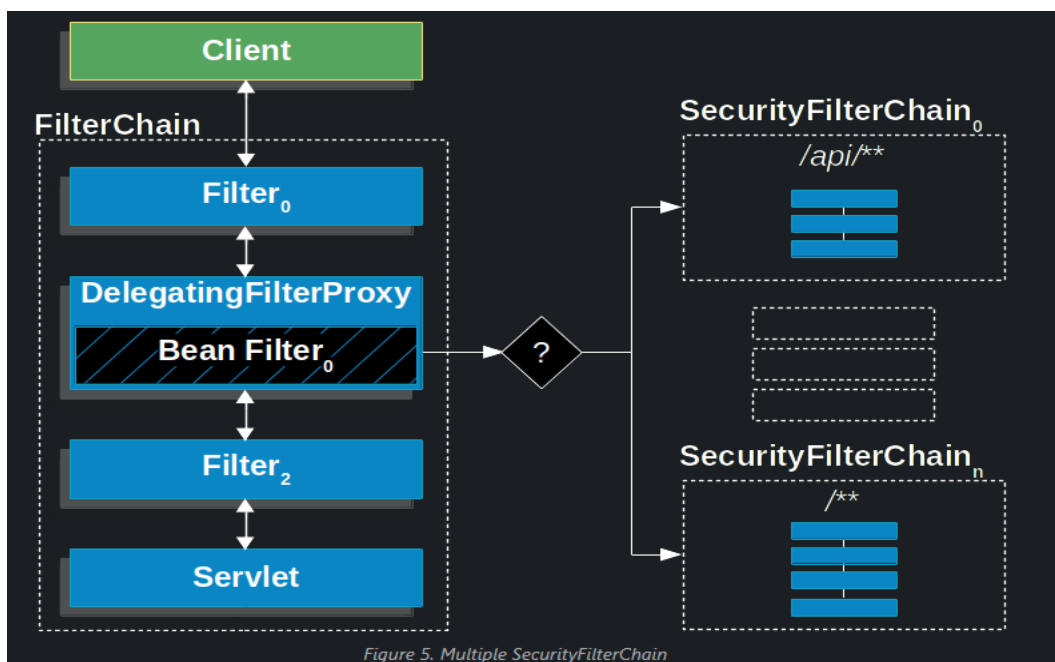
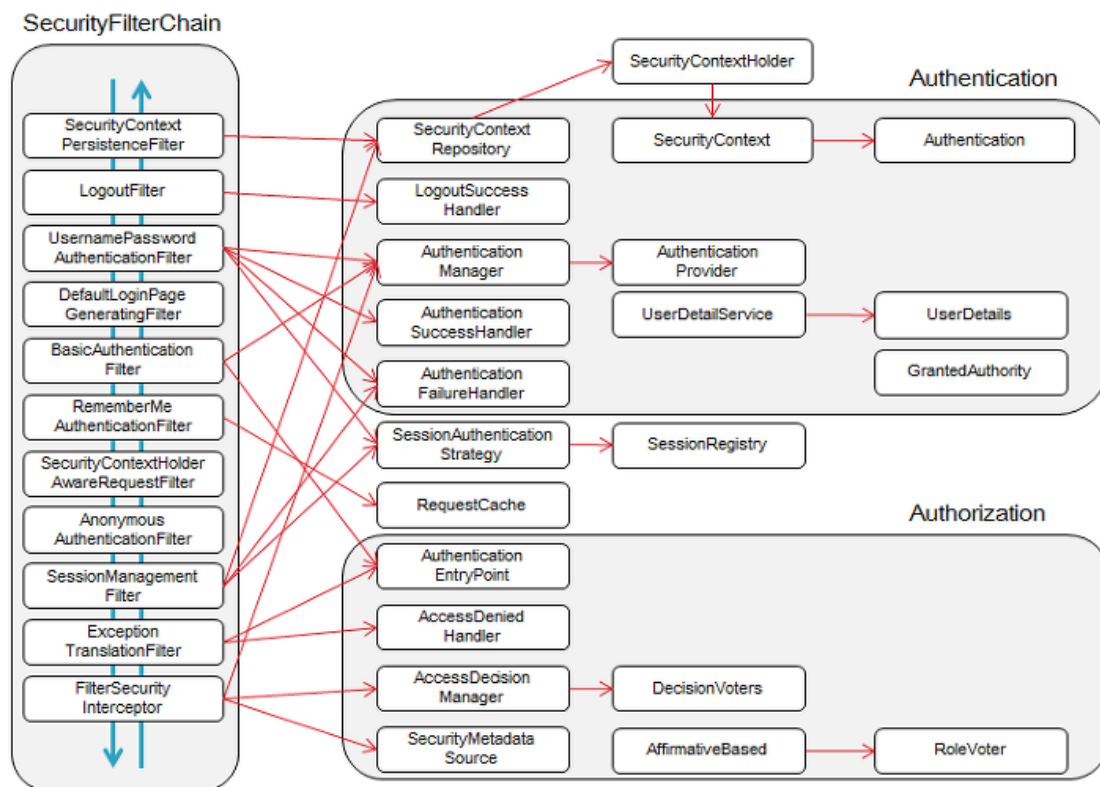


Figure 5. Multiple SecurityFilterChain



자동 설정에 의한 보안

Spring Security 관련 의존성이 추가된 것 만으로도 스프링 앱이 기동될 때 시큐리티 관련 설정이 자동으로 수행되어 기본 보안이 작동된다.

- 서버가 기동되면서 스프링 시큐리티의 초기화 작업 및 보안 설정이 이루어짐
- 별도의 설정이나 코드를 작성하지 않아도 기본적인 웹 보안 기능이 현재 시스템에 연동

- 1) 기본적으로 모든 요청에 대하여 인증여부를 검증하고 인증이 승인되어야 자원에 접근이 가능하다.
- 2) 인증 방식은 **폼 로그인 방식**과 **httpBasic 로그인 방식**을 제공한다.
- 3) 인증을 시도할 수 있는 로그인 페이지가 자동적으로 생성되어 렌더링 된다.
- 4) 인증 승인이 이루어질 수 있도록 한 개의 계정이 디폴트(SecurityProperties의 User 객체)로 제공된다.

사용자명 : user
패스워드 : 서버 기동 시 자동 생성(랜덤 문자열)

@Bean

@Order(SecurityProperties.BASIC_AUTH_ORDER)

```
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {  
    http.authorizeHttpRequests((requests) -> requests.anyRequest().authenticated());  
    http.formLogin(withDefaults());  
    http.httpBasic(withDefaults());  
    return http.build();  
}
```

SecurityBuilder / SecurityConfigurer

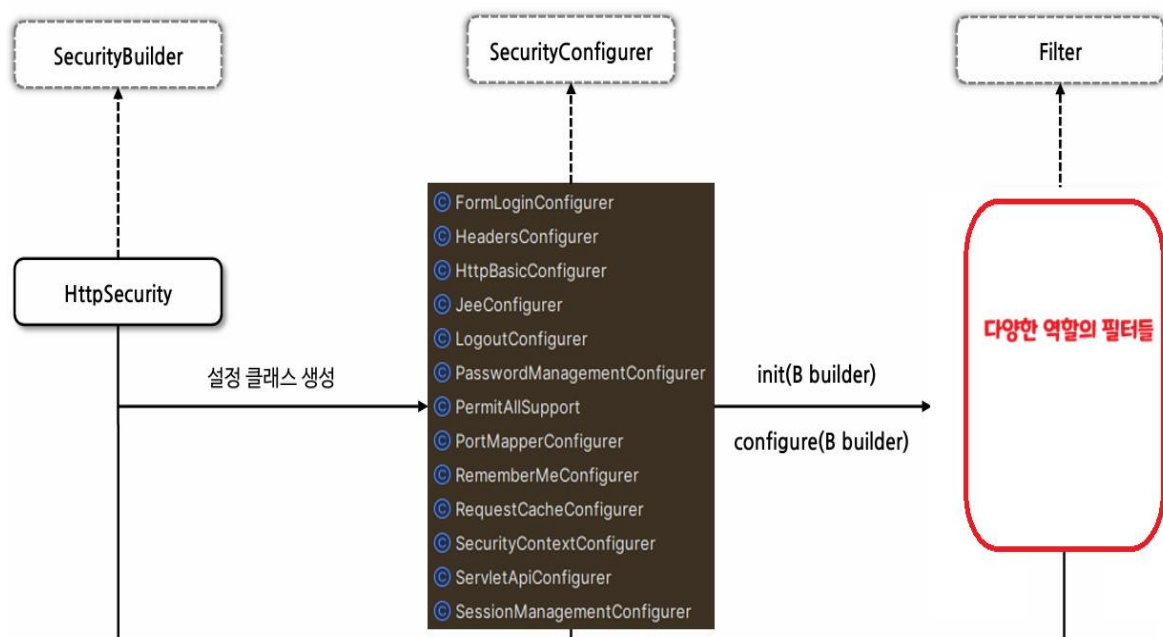
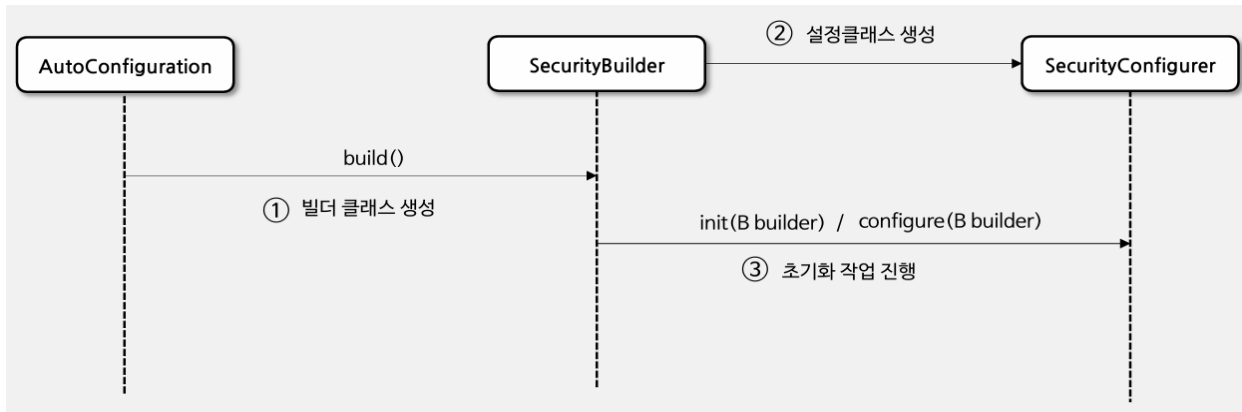
SecurityBuilder

- 빌더 클래스들의 최상위 인터페이스
- 웹 보안을 구성하는 빈 객체, 설정 클래스를 생성
- 구현클래스들로 WebSecurity, HttpSecurity 등이 있음

SecurityConfigurer

여러 초기화 설정 관여

- Http 요청과 관련된 보안처리를 담당하는 필터 생성
- 인증 및 인가 초기화 작업 설정에 관여
- 스프링 시큐리티는 모든 인증, 인가 요청을 필터를 통해서 함



HttpSecurity

- **HttpSecurityConfiguration** 에서 **HttpSecurity** 를 생성하고 초기화를 진행
- **HttpSecurity** 는 보안에 필요한 각 설정 클래스와 필터들을 생성하고 최종적으로 **SecurityFilterChain** 빈 생성

